Data Structures and Algorithm Analysis

Mark A. Weiss

Chapter 5 Hashing

5.9 Extendible Hashing

Our last topic in this chapter deals with the case where the amount of data is too large to fit in main memory. As we saw in Chapter 4, the main consideration then is the number of disk accesses required to retrieve data.

As before, we assume that at any point we have N records to store; the value of N changes over time. Furthermore, at most M records fit in one disk block. We will use $M = 4$ in this section.

If either probing hashing or separate chaining hashing is used, the major problem is that collisions could cause several blocks to be examined during a search, even for a well-distributed hash table. Furthermore, when the table gets too full, an extremely expensive rehashing step must be performed, which requires $O(N)$ disk accesses.

A clever alternative, known as extendible hashing, allows a search to be performed in two disk accesses. Insertions also require few disk accesses.

We recall from Chapter 4 that a B-tree has depth $O(\log_{M/2} N)$. As M increases, the depth of a B-tree decreases. We could in theory choose M to be so large that the depth of the B-tree would be 1. Then any search after the first would take one disk access, since, presumably, the root node could be stored in main memory. The problem with this strategy is that the branching factor is so high that it would take considerable processing to determine which leaf the data were in. If the time to perform this step could be reduced, then we would have a practical scheme. This is exactly the strategy used by extendible hashing.

Let us suppose, for the moment, that our data consists of several six-bit integers. **Figure 5.52** shows an extendible hashing scheme for these data. The root of the "tree" contains four links determined by the leading two bits of the data. Each leaf has up to $M = 4$ elements. It happens that in each leaf the first two bits are identical; this is indicated by the number in parentheses. To be more formal, D will represent the number of bits used by the root, which is sometimes known as the directory. The number of entries in the directory is thus 2^D. d_L is the number of leading bits that all the elements of some leaf L have in common. d_L will depend on the particular leaf, and $d_L \leq D$.

Suppose that we want to insert the key 100100. This would go into the third leaf, but as the third leaf is already full, there is no room. We thus split this leaf into two leaves, which are now determined by the first three bits. This requires increasing the directory size to 3. These changes are reflected in **Figure 5.53**.

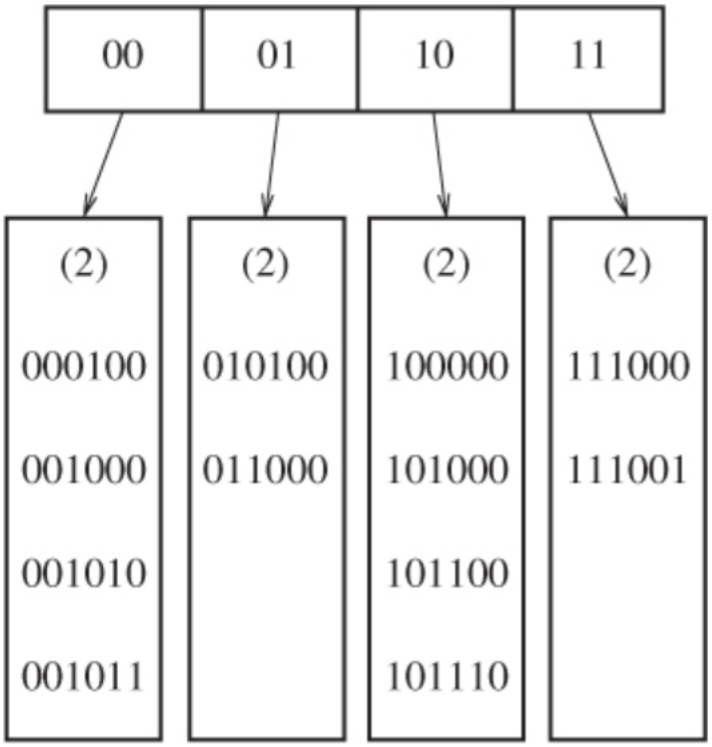

Figure 5.52 Extendible hashing: original data

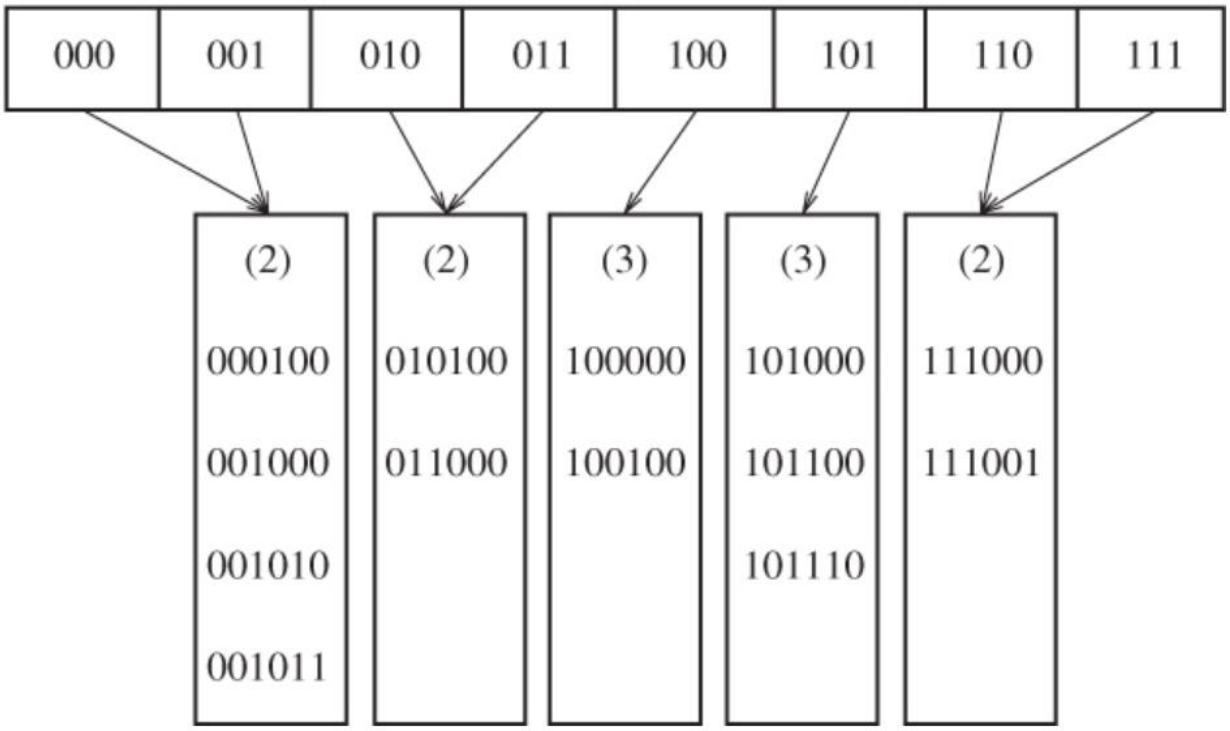

Figure 5.53 Extendible hashing: after insertion of 100100 and directory split

Notice that all the leaves not involved in the split are now pointed to by two adjacent directory entries. Thus, although an entire directory is rewritten, none of the other leaves are actually accessed.

If the key 000000 is now inserted, then the first leaf is split, generating two leaves with $d_L = 3$. Since $D = 3$, the only change required in the directory is the updating of the 000 and 001 links. See **Figure 5.54**.

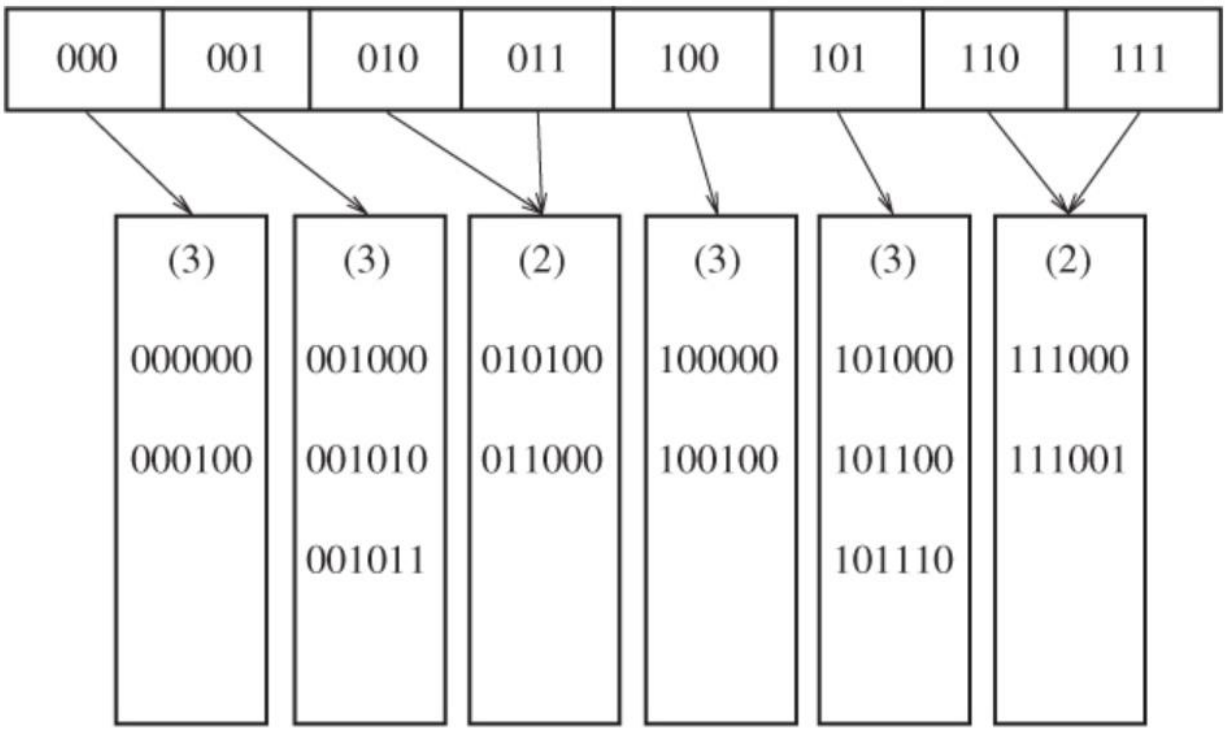

Figure 5.54 Extendible hashing: after insertion of 000000 and leaf split

This very simple strategy provides quick access times for insert and search operations on large databases. There are a few important details we have not considered.

First, it is possible that several directory splits will be required if the elements in a leaf agree in more than $D + 1$ leading bits. For instance, starting at the original example, with $D = 2$, if 111010, 111011, and finally 111100 are inserted, the directory size must be increased to 4 to distinguish between the five keys. This is an easy detail to take care of, but must not be forgotten. Second, there is the possibility of duplicate keys; if there are more than M duplicates, then this algorithm does not work at all. In this case, some other arrangements need to be made.

These possibilities suggest that it is important for the bits to be fairly random. This can be accomplished by hashing the keys into a reasonably long integer—hence the name.

We close by mentioning some of the performance properties of extendible hashing, which are derived after a very difficult analysis. These results are based on the reasonable assumption that the bit patterns are uniformly distributed.

The expected number of leaves is $(N/M) \log_2 e$. Thus the average leaf is $\ln 2 = 0.69$ full. This is the same as for B-trees, which is not entirely surprising, since for both data structures new nodes are created when the $(M + 1)$th entry is added.

The more surprising result is that the expected size of the directory (in other words, 2^D) is $O(N^{1+1/M}/M)$. If M is very small, then the directory can get unduly large. In this case, we can have the leaves contain links to the records instead of the actual records, thus increasing the value of M. This adds a second disk access to each search operation in order to maintain a smaller directory. If the directory is too large to fit in main memory, the second disk access would be needed anyway.